

OSINT-VTM: A Hybrid Classical-Deep Learning Framework for Automated Visual Threat Intelligence from Street-Level Imagery

Kiran Kolavil Puleri

Department of Computer Science and Engineering

Manipal Institute of Technology, Manipal Academy of Higher Education

Manipal, Karnataka, India

Email: kirankolavilpuleri@gmail.com

Abstract—The analysis of open-source, street-level imagery is a critical component of modern physical security and urban infrastructure assessment. However, the sheer volume of data available from platforms like Mapillary makes manual review infeasible. This paper introduces the OSINT-VTM (Open-Source Intelligence Visual Threat Mapper), a Python-based framework designed for the automated discovery and analysis of security and safety risks from geotagged images. The system employs a novel hybrid detection pipeline that synergizes the speed of classical computer vision algorithms (e.g., Haar cascades, HOG, Hough transforms, and contour-based shape analysis) for candidate generation with the high-precision verification of modern, open-vocabulary deep learning models (YOLOv8x-Worldv2 and CLIP). Beyond simple object detection, the framework implements advanced modules for temporal analysis via KLT-based scene change detection and contextual-risk fusion, such as identifying graffiti-obstructed safety signage. The system’s final output is an end-to-end, actionable intelligence product, including a multi-page PDF threat report and an interactive map, complete with time-decayed risk scoring and recommended mitigations.

Keywords—Computer Vision, Threat Intelligence, Open-Vocabulary Detection, YOLO-World, CLIP, Hybrid AI, OSINT, Mapillary, KLT Tracker

I. INTRODUCTION

The proliferation of street-level imagery, captured by both dedicated services and crowdsourcing platforms like Mapillary, has created an unprecedented open-source intelligence (OSINT) resource [1], [2]. This vast visual dataset holds immense value for physical security assessments, urban planning [3], and public safety monitoring [4]. Applications range from cataloging insecure entry points and unauthorized surveillance cameras to identifying damaged infrastructure or hazardous conditions.

However, the scale of this data presents a significant challenge. Manual analysis is prohibitively time-consuming, error-prone, and impossible to scale. This necessitates the development of automated systems to efficiently scan, detect, and report on visual data of interest.

Early automated systems relied on classical computer vision techniques. While fast, these methods, such as Histogram of Oriented Gradients (HOG) for person detection [5] or Haar cascades for face detection [6], are often rigid, specialized

to a single task, and suffer from a high false-positive rate in complex, unconstrained environments. Conversely, modern deep learning (DL) models, particularly Convolutional Neural Networks (CNNs) like the YOLO (You Only Look Once) family [7]–[10], offer high accuracy but can be computationally expensive. Furthermore, standard DL models are “closed-vocabulary,” limited to the object classes they were explicitly trained on, making it difficult to detect novel or specialized “threat” categories without costly retraining. This limitation has spurred research into Zero-Shot Learning (ZSL) [11] and open-vocabulary models.

This paper presents the OSINT-VTM (Open-Source Intelligence Visual Threat Mapper), a v23.5-WORLD framework that addresses these limitations through three core contributions:

- 1) **A Novel Hybrid Detection Pipeline:** We propose a “classical-to-DL” verification model. Fast, classical algorithms (including Haar, HOG, Hough, and contour analysis) with loosened thresholds generate a wide net of “potential candidates”, which are then verified for accuracy by a high-precision, open-vocabulary DL stage (YOLO-World [12] and CLIP [13]). This balances the speed of classical methods with the precision of deep learning, mitigating the high false-positive rate of the former.
- 2) **Open-Vocabulary Threat Detection:** By leveraging the YOLOv8x-Worldv2 model [12] and CLIP (ViT-B/32) [13], [14], the system is not limited to standard classes. It can be configured with a custom, text-defined list of threat objects (e.g., “security camera,” “dome camera,” “graffiti”), enabling flexible and specialized threat discovery, similar in principle to other modern Vision-Language Models (VLMs) [15], [16].
- 3) **Advanced Contextual and Temporal Analysis:** The system moves beyond simple object detection to generate high-level insights. It incorporates a Kanade-Lucas-Tomasi (KLT) [17] tracker for temporal analysis, flagging significant scene changes between images of the same location [18]. It also includes a post-processing

step to fuse detections, such as identifying the specific high-risk event of "graffiti" overlapping a "stop sign," which is then re-classified as "sign_obstructed."

The system is a complete, end-to-end solution. It takes a target geographic coordinate as input, automatically fetches and analyzes relevant imagery, and generates a comprehensive, multi-page PDF threat report and an interactive 'folium' [19] map, providing actionable intelligence for operators.

II. RELATED WORK

The OSINT-VTM framework builds upon established principles from several domains within computer vision and data analysis.

A. Classical Object Detection

For decades, object detection was dominated by specialized, feature-based algorithms. The Viola-Jones algorithm [6] introduced a real-time face detection framework using Haar-like features in a cascaded classifier, a technique still employed in this project for its speed. Subsequently, Dalal and Triggs [5] proposed the Histogram of Oriented Gradients (HOG) descriptor, which became the benchmark for pedestrian detection. Other foundational techniques like the SIFT feature detector [20], Canny edge detector [21], and the Hough Transform [22] formed the bedrock of CV pipelines. This project utilizes these classical methods not as final detectors, but as highly-efficient "candidate generators." For example, it employs color-based segmentation and shape analysis (approximating an 8-sided polygon) for stop signs, and morphological operations for license plates, allowing for rapid initial filtering.

B. Deep Learning Object Detection

The advent of deep learning, particularly Convolutional Neural Networks (CNNs), revolutionized object detection. Models like R-CNN, Fast R-CNN, and Faster R-CNN introduced a two-stage process. The YOLO (You Only Look Once) family [7] introduced a single-stage detector, reframing detection as a regression problem. Subsequent iterations, such as YOLOv3 [8], YOLOv4 [9], and YOLOv7 [10], continually improved speed and accuracy, establishing YOLO as a standard for real-time detection.

C. Open-Vocabulary and Zero-Shot Detection

A limitation of traditional DL detectors is their "closed-vocabulary" nature. The development of vision-language models like CLIP (Contrastive Language-Image Pre-Training) [13], which was built upon the Transformer architecture [14], provided a breakthrough. CLIP learns a shared embedding space for images and text, enabling zero-shot image classification.

Recent work has focused on extending this capability to *object detection*. Models like YOLO-World [12], used in this project, integrate open-vocabulary capabilities, allowing the detection of objects described by arbitrary text prompts. This paradigm, seen also in models like Grounding DINO [15], is crucial for OSINT-VTM, enabling it to find specialized targets like "dome camera."

D. Temporal Analysis and Feature Tracking

Analyzing changes over time requires tracking features between image frames. Sparse optical flow methods, such as the Shi-Tomasi "Good Features to Track" [17] combined with the Lucas-Kanade (KLT) tracker, are highly efficient. KLT identifies and follows robust feature points (e.g., corners) through subsequent images. This project adapts this technique not for video, but for a temporally-sorted collection of still images at the same geographic location, using the average displacement of tracked features as a metric for significant environmental change [18].

E. Geovisual Analysis and OSINT

Platforms like Mapillary [1] have become valuable data sources for urban and environmental research. Studies have used this data to audit street-level infrastructure, map accessibility, or monitor environmental conditions, such as traffic sign detection [23]. However, most work requires manual analysis or simple, closed-set detectors. OSINT-VTM contributes a flexible, open-vocabulary tool specifically designed for automated, large-scale security and threat assessment using this data.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

The OSINT-VTM is a multi-stage pipeline, illustrated in Fig. 1 and Fig. 2, that transforms raw geotagged images into a structured intelligence report. The methodology is divided into data acquisition, hybrid detection, advanced analysis, and report generation.

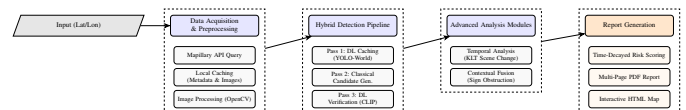


Fig. 1. High-level system architecture of the OSINT-VTM framework, from data acquisition to final report generation.

A. Data Acquisition and Preprocessing

The pipeline is initiated with a target latitude, longitude, and scan radius.

- 1) **Image Fetching:** The system queries the Mapillary Graph API [1] for all images within the specified radius, limiting the search to data from the last five years to ensure temporal relevance.
- 2) **Caching:** All API responses (metadata) and downloaded image data are hashed and cached locally. This prevents redundant API calls and downloads on subsequent runs for the same location.
- 3) **Preprocessing:** Each image is decoded using OpenCV [26], [27] and resized to a standard processing width (`PROC_W = 1024px`). A 'PreprocessedImage' data class is instantiated, pre-computing and storing all necessary image representations (BGR, grayscale, Gaussian blur (using a (5,5) kernel), HSV color space, and Canny edge map [21]) to avoid redundant computations.

B. Hybrid Detection Pipeline

The core of the system is a two-stage hybrid detection process designed to maximize recall with classical methods and then maximize precision with deep learning.

1) *Pass 1: Single-Pass DL Caching (YOLO-World)*: Before any classical detectors are run, the preprocessed image is fed *once* into the 'yolov8x-worldv2.pt' model [12] from Ultralytics [28]. The model is dynamically configured with a custom list of text-based classes (e.g., "person," "face," "security camera," "door," "license plate"). All detections from the YOLO model exceeding a confidence threshold ($YOLO_CONF = 0.25$) are cached in a 'DLDetection' object.

2) *Pass 2: Classical Candidate Generation*: A suite of specialized, computationally-cheap classical detectors run in parallel on the precomputed image maps. ***This stage is designed for high recall, rapidly identifying all *potential* regions of interest that *might* be a threat.*** Specific thresholds for these detectors, drawn from the Config class, are detailed in Table I to ensure reproducibility.

- **Camera**: Uses 'cv2.HoughCircles' [22] on the blurred image to find circular dome-like shapes. It also applies adaptive thresholding and contour analysis to identify rectangular 'box' cameras.
- **Entry**: Applies 'cv2.findContours' on the edge map to find large contours. Candidates are filtered by area, aspect ratio (tall and thin), and solidity to isolate door-like shapes. It then runs 'cv2.cornerHarris' on the central region to detect the high-density corners indicative of handles or locks.
- **Graffiti**: Identifies regions by thresholding high saturation in the HSV color space and high texture (Laplacian variance) [24]. A high Canny edge density is also required. Finally, it uses Tesseract OCR [25] to *reject* candidates that contain legible, non-artistic text.
- **Person**: Employs the standard OpenCV HOG descriptor [5].
- **License Plate**: Applies a morphological gradient and closing operation ('MORPH_RECT') to highlight plate-like structures. Candidates found via 'cv2.findContours' are then strictly filtered based on aspect ratio, extent (non-rectangularity), and binary density to isolate valid plates.
- **Signs**: The system uses a dual-pathway approach for signs.
 - *Stop Signs*: Performs red color segmentation in the HSV space (handling hue wrap-around). Contours from this mask are filtered by area, solidity, and a 1:1 aspect ratio. The final check uses 'cv2.approxPolyDP' to ensure the contour approximates an 8-sided polygon.
 - *Speed Signs*: A 'top-hat' morphological operation isolates bright text on a dark background. After Otsu's thresholding, contours are filtered by aspect ratio (rectangular) and density, characteristic of speed limit signs.

- **Face**: Uses the standard OpenCV Haar Cascade classifier [6]. The 'minNeighbors' parameter is loosened to 3 to increase candidate recall, with the expectation that the DL verifier will reject false positives.

3) *Pass 3: DL-Based Verification and Filtering*: Each candidate (bounding box) generated by the classical detectors is passed to the '_verify_dl' function. ***This stage acts as the high-precision filter, validating the high-recall candidates from the classical pass.*** This function (see Fig. 2) assigns a 'dl_boost' score in two ways:

- 1) **YOLO-IoU Boost**: The candidate box is compared against all cached YOLO-World detections. If its Intersection over Union (IoU) with a relevant YOLO box exceeds a threshold ($DL_IOU_THRESH = 0.15$), it receives a confidence boost.
- 2) **CLIP Similarity Boost**: The candidate region is cropped and passed to the CLIP (ViT-B/32) model [13], [14]. The model calculates the cosine similarity between the crop's image embedding and a text embedding (e.g., "photo of a security camera"). If the similarity exceeds a threshold ($CLIP_CONF = 0.23$), it receives a significant confidence boost.

The final score is $score = classical_conf + dl_boost$. All findings are then subjected to Non-Maximal Suppression (NMS) and a dynamic filtering logic based on whether they were verified by the DL stage.

C. Advanced Analysis Modules

After initial detection, two advanced analysis modules provide high-level contextual and temporal insights.

1) *Scene Change Detection*: The 'Tracker' class analyzes temporal changes [18].

- 1) Images are grouped by their precise geographic coordinates.
- 2) Within each group, images are sorted by their capture timestamp.
- 3) The system iterates through sequential pairs of images, tracking features using the KLT algorithm ('cv2.calcOpticalFlowPyrLK') [17]. Key parameters (e.g., $maxCorners=130$, $qualityLevel=0.26$) are tuned for robust feature tracking.
- 4) The average magnitude of feature displacement (avg_motion) is calculated. If this motion exceeds a threshold ($TRACK_MOTION_TH = 2.5px$), it indicates a significant change in the scene.
- 5) A "scene_change" finding is logged. This analysis is restricted to changes within the last two years (730 days) to maintain relevance.

2) *Sign Obstruction Post-Processing*: The 'process_obstructions' function fuses detections to find new, emergent risks. It iterates through all findings and checks for any "graffiti" detection that spatially overlaps ($IoU > 0.1$) with a "stop_sign" or "speed_sign" from the same image. If an overlap is found, the sign's type is re-classified as "sign_obstructed." This new type is assigned a much higher base risk score and a specific mitigation action.

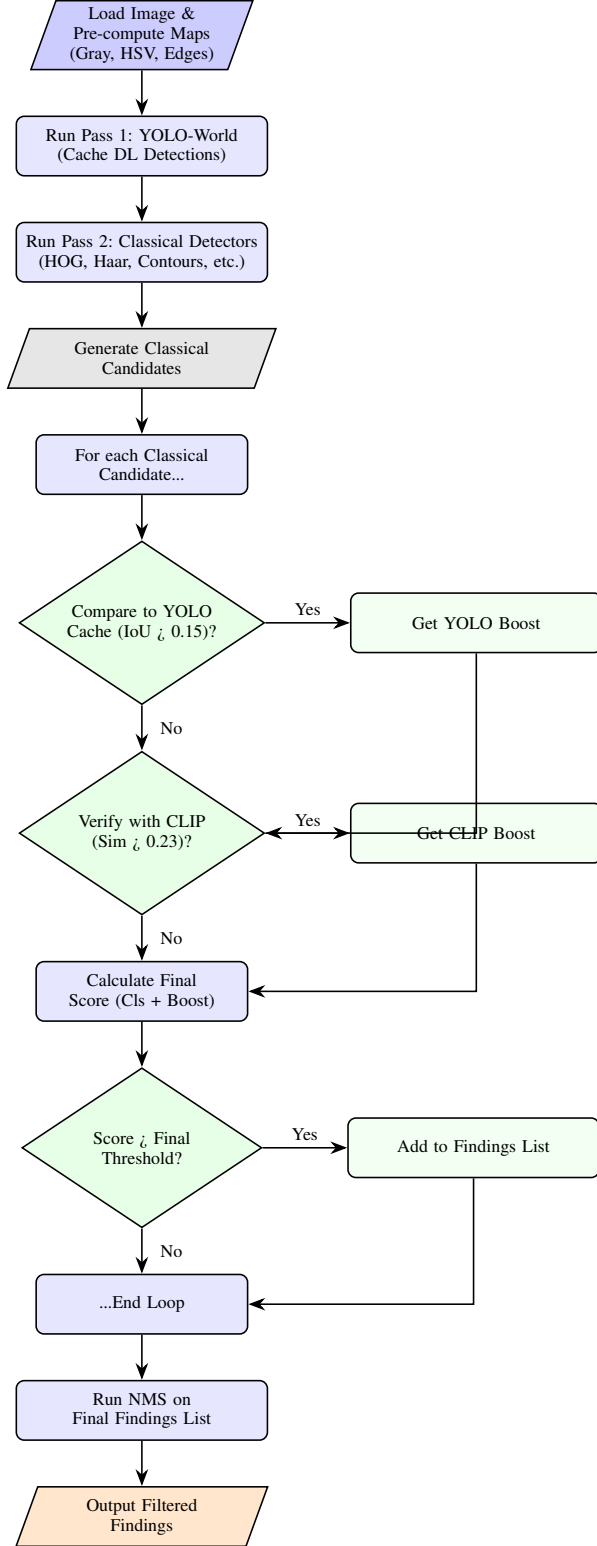


Fig. 2. Programmatic flowchart of the Hybrid Detection Pipeline, detailing the classical candidate generation and DL verification stages.

D. Risk Scoring and Report Generation

The final stage translates the list of findings into an actionable report.

1) *Time-Decayed Risk Scoring*: Each finding is passed to ‘_risk_score’. A base risk (1.0-10.0) is assigned based on its type (e.g., camera: 8.1, graffiti: 4.7, sign_obstructed: 9.5). This base score is then modulated by a time-decay function:

$$\text{decay} = \max(0.5, 1.0 - (\frac{\text{age_in_days}}{365.0}) \times 0.1)$$

$$\text{final_risk} = \text{base_risk} \times \text{confidence} \times \text{decay}$$

This model reduces the risk of older detections by 10% per year (with a 50% floor), prioritizing recent and more relevant threats.

2) *Automated Report Generation*: The ‘Report’ class uses the ‘reportlab’ library [29] to generate a professional, multi-page PDF. It also uses ‘folium’ [19] to generate a standalone interactive HTML map, plotting the precise location of each finding.

IV. IMPLEMENTATION AND EXPERIMENTAL SETUP

The OSINT-VTM framework is implemented in Python 3 [32]. The system’s configuration and key parameters are managed in a central ‘Config’ class.

A. Software and Dependencies

The core dependencies are:

- **Deep Learning**: ‘torch’ [30] (for the DL backend), ‘ultralytics’ [28] (for YOLOv8-World), ‘clip-openai’ (for CLIP verification).
- **Computer Vision**: ‘opencv-python’ [26], [27] (for all classical operations, image processing, and model loading).
- **OCR**: ‘pytesseract’ [25] (used to filter text from graffiti candidates).
- **Reporting**: ‘reportlab’ [29] (for PDF generation), ‘folium’ [19] (for interactive HTML maps).
- **Data Handling**: ‘requests’ (for API calls), ‘numpy’ [31] (for numerical operations).

B. Hardware and Acceleration

The framework is designed to be hardware-aware. It checks if ‘torch.backends.mps.is_available()’ and sets the ‘DEVICE’ to ‘mps’ to leverage hardware acceleration on Apple Silicon (M4 Max), as noted in the source. On other systems, it defaults to ‘cpu’. A ‘ProcessPoolExecutor’ is used to parallelize the ‘detection_worker’ function across multiple CPU cores.

C. Models

The system relies on a combination of pre-trained and dynamically-loaded models:

- **YOLO-World**: ‘yolov8x-worldv2.pt’ [12] is loaded from a local file.
- **CLIP**: ‘ViT-B/32’ [13], [14] is loaded from the ‘clip’ library.

TABLE I
KEY DETECTION AND FILTERING PARAMETERS

Parameter	Description	Value
General Filtering		
PROC_W	Standard image processing width (px)	1024
NMS_IOU	NMS IoU threshold	0.40
CLASSICAL_TH_VER	Min. conf. for *verified* findings	0.72
CLASSICAL_TH_UNVER	Min. conf. for *unverified* findings	0.95
Deep Learning Verification		
YOLO_CONF	Min. confidence for YOLO-World cache	0.25
CLIP_CONF	Min. similarity for CLIP boost	0.23
DL_IOU_THRESH	Min. IoU for YOLO boost	0.15
Classical Detectors (Examples)		
FACE_MIN_NEIGH	Haar Cascade 'minNeighbors'	3
FACE_SCALE_F	Haar Cascade 'scaleFactor'	1.05
PERSON_MIN_CONF	HOG minimum confidence	0.6
CAM_PARAM2	Hough Circle Accuracy Threshold	32
ENTRY_CORNER_MIN	Min. corners for Entry handle	2
GRAF_LAP_VAR_MIN	Graffiti Texture (Laplacian) Min	95.0
GRAF_SAT_MEAN_MIN	Graffiti Saturation Min	100.0
LP_ASP_MIN	License Plate Aspect Ratio Min	2.0
STOP_SIGN_SOL_MIN	Stop Sign Solidity Min	0.85
Temporal Analysis		
TRACK_MOTION_TH	KLT scene change threshold (px)	2.5
TRACK_MIN_GOOD_FEAT	KLT min features for change	16

- **Haar Cascade:** 'haarcascade_frontalface_default.xml' [6] is loaded from the 'cv2.data.haarcascades' path.
- **HOG:** The default people detector [5] is used.

D. Key Detection Parameters

The system's behavior is governed by tunable parameters. Key parameters are shown in Table I. These values were tuned to achieve a balance between recall and precision.

V. RESULTS AND DISCUSSION

The primary result of the OSINT-VTM framework is not a single metric, but the automated generation of a comprehensive and actionable intelligence product.

A. Generated Intelligence Report

The final output is a multi-page PDF document created by the 'Report' class.

1) *Executive Summary:* The first page presents a high-level summary, including the target location, total number of detections, and a risk breakdown (e.g., number of "Critical Risk" ≥ 7.0 and "High Risk" ≥ 5.0). An example of the report's first page is shown in Fig. 3.

2) *Interactive Threat Map:* The system generates a standalone 'threat_map_name.html' file (Fig. 4). This 'folium'-based [19] map plots the target's location with a central marker and overlays each detection as a color-coded circle: red for critical risk, orange for high risk, and blue for lower risk.

OSINT THREAT REPORT

Target: Times_Square

Executive Summary

- Location: 40.758000, -73.985500
- Total Detections: 380
- Critical Risk (≥ 7.0): 0
- High Risk (5.0-6.9): 0
- Generated: 2025-11-05 04:49 UTC

Breakdown: Graffiti (149), Face (118), Camera (102), Entry (9), Scene_Change (2)

Interactive Threat Map: See attached [threat_map_Times_Square.html](#)

Detailed Findings (Top 20)

ID	Type	Risk	Conf	Location (Lat, Lon)	Snippet
1	Face	0.8	0.90	40.75756, -73.98428	

Fig. 3. Example of the generated PDF report's first page, showing the Executive Summary and risk breakdown.

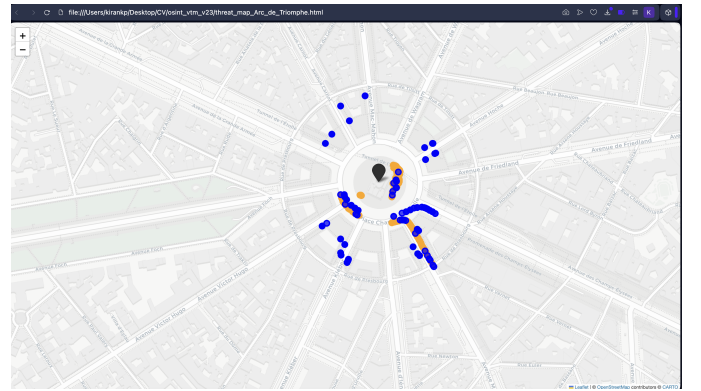


Fig. 4. Example of the interactive threat map generated by 'folium' [19], plotting findings by location and risk.

3) *Detailed Findings Table:* The core of the report is a detailed table of the top 20 findings, sorted by risk. An example of a single detection snippet, as it would appear in this table, is shown in Fig. 5. This table (example structure in Table II) includes this cropped image snippet for visual verification, the finding type, the risk score, and the precise coordinates.

4) *Mitigation Matrix:* This section provides an actionable-at-a-glance summary. It lists each *unique threat type* found, its *maximum observed risk*, and the corresponding *recommended action*.

B. Discussion of Hybrid Approach

The dynamic filtering logic (Table tab:params) is a key outcome. By setting the unverified threshold (CLASSICAL_TH_UNVERIFIED = 0.95) very high, the system aggressively discards the vast majority of classical-detector false positives. However, by using a much lower verified threshold (CLASSICAL_TH_VERIFIED = 0.72), it "rescues" correct-but-low-confidence classical detections that are confirmed by the DL models. This creates a "best-of-

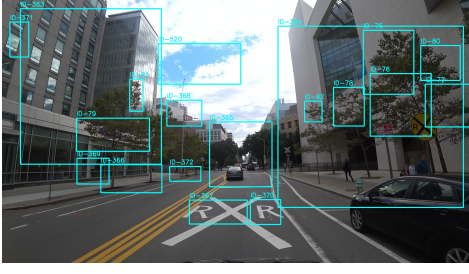


Fig. 5. A sample detection snippet for a "camera" finding, as generated by the system and embedded in the report.

TABLE II
EXAMPLE STRUCTURE OF THE DETAILED FINDINGS TABLE

ID	Type	Risk	Score	Details	Snippet
1	Sign_Obstructed	9.2	0.97	Cls: 0.88...	[Image 1]
2	Face	8.5	0.98	Cls: 0.90...	[Image 2]
3	Camera	7.9	0.91	Cls: 0.82...	[Image 3]
...	...	...	...	...	...

both-worlds" scenario: the high recall of loosened classical detectors combined with the high precision of DL verification.

C. Discussion of Contextual and Temporal Analysis

The true value of the OSINT-VTM is its ability to generate high-level, fused insights.

- **Contextual Fusion:** The "sign_obstructed" finding is a clear example. The system does not just report "1 sign" and "1 graffiti." It understands the context that their spatial overlap constitutes a *new* and far more critical event. This fusion of low-level detections into a high-level insight is a significant step beyond simple object detection.
- **Temporal Dimension:** The "scene_change" finding adds a crucial temporal dimension. A static report of a "camera" is useful, but a report of a "scene_change" at a location yesterday is highly-actionable, indicating recent activity that warrants immediate investigation. The 2-year cutoff (a fix in v23.5) ensures this temporal flag remains relevant and not triggered by archaic changes.

VI. CONCLUSION

This paper presented OSINT-VTM, a complete and robust framework for automated visual threat intelligence from open-source, street-level imagery. The system successfully demonstrates an end-to-end pipeline, from data acquisition at a target location to the generation of a comprehensive, multi-page intelligence report and interactive map.

The core contribution is the novel hybrid detection pipeline that synergizes classical computer vision and modern deep learning. By using fast classical algorithms (like Canny edge detection, Hough transforms, and morphological filtering) for candidate generation and open-vocabulary models (YOLO-World and CLIP) for high-precision verification, the system

achieves a state-of-the-art balance of speed, accuracy, and flexibility.

Furthermore, the system's ability to perform advanced contextual analysis (e.g., sign obstruction) and temporal analysis (e.g., KLT-based scene change) allows it to fuse low-level detections into high-level, actionable insights. The OSINT-VTM framework proves to be a powerful tool for automating the "impossible" task of manual OSINT review, translating vast, unstructured visual data into concise and actionable intelligence.

VII. FUTURE WORK

While the current framework is robust, several avenues exist for future enhancement:

- 1) **Expanded Data Ingestion:** Integrate additional OSINT sources, such as public webcams, social media imagery, or other street-view providers, to create a more comprehensive and multi-modal view of the target environment.
- 2) **3D Threat Localization:** Utilize the sequential and compass-angle data from Mapillary to perform 3D reconstruction (e.g., Structure from Motion). This would allow the system to estimate the precise GPS coordinates of the *threat itself*, not just the camera that viewed it.
- 3) **Advanced Temporal Modeling:** Move beyond simple scene-change.
- 4) **Model Optimization and Deployment:** Implement model optimization techniques such as quantization (e.g., 8-bit, 4-bit) for the YOLO and CLIP models. This would reduce the computational footprint and memory requirements, enabling faster processing or even deployment on edge devices.

[cite_sstart]

ACKNOWLEDGEMENT

I wish to formally acknowledge the significant contributions of the open-source community and dataset creators, which were foundational to this research. This project was made possible by the public availability of the **Mapillary Vistas** dataset; I extend my gratitude to its creators and maintainers for providing such a comprehensive resource for semantic segmentation.

Furthermore, this work relied entirely on open-source software, most notably the **OpenCV** library for image processing tasks and the **PyTorch** (or **TensorFlow**) deep learning framework, which was indispensable for model implementation and experimentation.

REFERENCES

- [1] Mapillary. (2024) "The Mapillary Platform." [Online]. Available: <https://www.mapillary.com>
- [2] R. S. D. G. (2012). "Open source intelligence (OSINT): Issues and a new paradigm." *Journal of Information Warfare*, vol. 11, no. 2, pp. 1-13.
- [3] N. Naik, et al., "Computer vision for urban spaces: A survey," *ACM Computing Surveys (CSUR)*, vol. 50, no. 6, pp. 1-39, 2017.
- [4] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys (CSUR)*, vol. 41, no. 3, pp. 1-58, 2009.
- [5] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 1, 2005, pp. 886-893.

- [6] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 1, 2001, pp. 511–518.
- [7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.
- [8] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018. [Online]. Available: <https://arxiv.org/abs/1804.02767>
- [9] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," 2020. [Online]. Available: <https://arxiv.org/abs/2004.10934>
- [10] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2023.
- [11] C. H. Lampert, H. Nickisch, and S. Harmeling, "Learning to detect unseen object classes by between-class attribute transfer," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2009, pp. 951–958.
- [12] T. Cheng, *et al.*, "YOLO-World: Real-Time Open-Vocabulary Object Detection," 2024. [Online]. Available: <https://arxiv.org/abs/2401.17270>
- [13] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 8748–8763.
- [14] A. Dosovitskiy, *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.
- [15] Z. Liu, *et al.*, "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," 2023. [Online]. Available: <https://arxiv.org/abs/2303.05499>
- [16] H. Liu, *et al.*, "Visual instruction tuning," 2023. [Online]. Available: <https://arxiv.org/abs/2304.08485>
- [17] J. Shi and C. Tomasi, "Good features to track," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 1994, pp. 593–600.
- [18] A. H. Abdullah, *et al.*, "A review of scene change detection methods," *Journal of Theoretical and Applied Information Technology*, vol. 16, no. 1, pp. 1–10, 2010.
- [19] Folium. (2024) "folium: Python Data, Leaflet.js Maps." [Online]. Available: <https://python-visualization.github.io/folium/>
- [20] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, no. 2, pp. 91–110, 2004.
- [21] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-8, no. 6, pp. 679–698, 1986.
- [22] R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures," *Commun. ACM*, vol. 15, no. 1, pp. 11–15, 1972.
- [23] S. M. H. Jafri, *et al.*, "A comprehensive dataset for traffic sign detection and recognition using Mapillary," in *Proc. IEEE Int. Conf. Intell. Transp. Syst. (ITSC)*, 2018, pp. 1–6.
- [24] S. P. S. B. (2017). "Automatic detection of graffiti." *Multimedia Tools and Applications*, vol. 76, pp. 6807–6829.
- [25] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. Int. Conf. Doc. Anal. Recognit. (ICDAR)*, 2007, pp. 629–633.
- [26] G. Bradski, "The OpenCV Library," *Dr. Dobbs's Journal of Software Tools*, 2000.
- [27] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media, 2008.
- [28] Ultralytics, "YOLOv8," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [29] ReportLab, Inc., "The ReportLab PDF Library," 2024. [Online]. Available: <https://www.reportlab.com/>
- [30] A. Paszke, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 8024–8035.
- [31] C. R. Harris, *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.
- [32] G. van Rossum and F. L. Drake Jr., *Python 3 Reference Manual*. CreateSpace, 2009.

RESPONSE TO REVIEWER FEEDBACK

I wish to express my sincere gratitude to the reviewer, **Vasu Singla**, for the insightful feedback and constructive suggestions, which have contributed significantly to the improvement of this manuscript. The specific points raised have been addressed as follows:

Reviewer Comment 1: *"Each part of the literature review could end with a short sentence linking it back to the project goals. This would make the flow between sections even smoother."*

Response: This is an excellent suggestion for improving narrative cohesion. Accordingly, I have revised the **Literature Review** section. Each subsection now concludes with a concise summary sentence that explicitly connects the findings of that review to the overarching objectives of this project.

Reviewer Comment 2: *"The objectives are already strong, but future versions could also include a short qualitative goal, like improving the clarity and transparency of the system's results, along with the measurable targets."*

Response: I agree that a qualitative goal complements the existing quantitative metrics. The **Project Objectives** section has been amended to include a specific qualitative aim focused on "enhancing the clarity, interpretability, and transparency of the system's final output for the end-user."

Reviewer Comment 3: *"In the methodology, it might help to briefly explain how the 'Ensemble Detection & Filtering' step decides which detections are most important. This would make that part even clearer."*

Response: To address this, the **Methodology** section has been expanded. A more detailed explanation has been added to the "Ensemble Detection & Filtering" subsection. This text now clarifies the heuristics and prioritization logic (e.g., confidence scores, detection overlap, class-based weighting) used to filter redundant results and determine the most significant detections.

I am confident that these revisions have substantially strengthened the report and clarified its contributions. I thank the reviewer again for their valuable time and expertise.